Trabajo Práctico 3: Optimización de Modelos de Machine Learning

Materia: 72.75 Aprendizaje Automático (Machine Learning)

Fecha de defensa: 11/11/26 y 18/11/26

En este trabajo práctico se abordará un problema de clasificación supervisada mediante un proceso iterativo de evaluación y optimización de modelos de Machine Learning. El objetivo principal es explorar y comparar diferentes estrategias de mejora, incluyendo selección de modelos, optimización de hiperparámetros, ingeniería de características entre otros métodos, para maximizar el rendimiento predictivo y evaluar la capacidad de generalización del sistema.

La presentación durará 10 minutos + 10 minutos de preguntas.

*mandar la presentación, el predictor y el código 24 horas antes de la clase en la que se presentará el TP

---

0. Dataset

Para este trabajo se utilizará un dataset relacionado con reservas de infraestructura GPU en un entorno de cómputo distribuido. El objetivo es predecir si una reserva será cancelada antes de su ejecución (canceled_job).

El dataset contiene información relacionada con características temporales de la reserva, tipo de recursos solicitados, historial del cliente, duración y complejidad del workload y características económicas y contractuales de la reserva.

El dataset principal pueden encontrarlo dentro de la carpeta **Competición** en campus, con el nombre de “gpu_resource_reservations”.

**Para más información sobre las variables del dataset, ver páginas 4 y 5 de este documento.*

1. Preprocesamiento y análisis exploratorio

Aunque no es el foco principal del TP, se valorará positivamente que se apliquen (y mencionen en la presentación) técnicas básicas vistas previamente de:

- limpieza de datos,
- tratamiento de missing values,
- encoding de variables categóricas,
- escalado,
- análisis exploratorio,
- y análisis de cohortes o subgrupos.
- **Pueden tomar cualquier decisión sobre los datos, como eliminar/seleccionar/combinar/transformar alguna feature o datos, pero justifiquen la decisión**

Además, recuerda realizar la división de datos entre entrenamiento y test antes de aplicar cualquier técnica que pueda producir data leakage.

2. Model-centered Machine Learning

2.1 Clasificadores - Establecimiento del baseline

Implementar (usando Scikit-learn) al menos uno de los clasificadores vistos en clase y hacer una breve calibración de hiperparámetros para establecer un clasificador baseline razonable.

2.2 Validación cruzada

Para la evaluación de estos clasificadores, implementar un esquema de k-fold cross-validation utilizando únicamente el conjunto de entrenamiento.

2.3 Métricas

Selecciona dos métricas vistas en clase y justifica su elección.

Utiliza estas métricas para:

- comparar modelos,
- seleccionar hiperparámetros,
- y tomar decisiones sobre el modelo baseline final.

3. Error analysis y enfoque data-centered

3.1 Análisis de errores

Utilizando el modelo baseline que consideren más adecuado en base a los análisis anteriores, realizar un análisis detallado de errores.

Se espera:

- definición de cohortes relevantes en los datos (en particular de *client_type, service_tier, reserved_gpu_type)*
- análisis por cohortes o subgrupos,
- análisis de falsos positivos y falsos negativos,
- identificación de regiones problemáticas del dataset,

3.2 Curvas de aprendizaje

Analizar cómo evoluciona el rendimiento del modelo al aumentar la cantidad de datos de entrenamiento.

Discutir:

- si el sistema parece limitado por el modelo o por los datos,
- y si obtener más datos podría mejorar significativamente el rendimiento.

3.3 Estrategia data-centered

A partir del análisis de errores del apartado 3.1, seleccionar una cohorte o subconjunto problemático/prometedor de datos y justificar por qué consideran prioritario intervenir sobre él.

Se deberá discutir:

- frecuencia del problema,
- impacto operativo,
- y posibles causas del mal rendimiento
- propuesta de siguientes pasos y posibles áreas o estrategias para mejorar

3.4 Adquisición de datos

Cada grupo deberá decidir qué datos adicionales incorporaría y justificar por qué esperan que eso mejore el sistema.

4. Optimización del sistema de Machine Learning

A partir de los resultados del baseline y del diagnóstico realizado, desarrollar y evaluar estrategias para mejorar el rendimiento predictivo del sistema.

Se espera que cada grupo explore al menos dos estrategias de mejora, que pueden incluir:

-Feature engineering y selección de variables (creación de nuevas features, transformaciones, selección o eliminación de variables)

-Model selection: comparación de diferentes familias de clasificadores (Random Forest, SVM, Gradient Boosting, etc.).

-Hyperparameter optimization: aplicación de técnicas de búsqueda y optimización de hiperparámetros.

-Ensemble learning: implementación de técnicas como bagging, boosting, voting o stacking.

-Data-centered improvements: estrategias de tratamiento de clases desbalanceadas, selección de ejemplos o incorporación de datos adicionales.

Para cada estrategia implementada, deberán:

1. Justificar la modificación propuesta, preferentemente a partir de los análisis anteriores.
2. Evaluar su impacto mediante un esquema de validación consistente.
3. Comparar los resultados con el baseline.

5. Modelo final

Seleccionar y documentar la estrategia final que consideren más adecuada.

Evaluar el sistema final y estimar cuál es la performance esperada en datos nuevos. Discutir especialmente qué cambios produjeron mayores mejoras y si esas mejoras fueron globales o específicas de ciertos subgrupos.

Crear y submitir (por mail) la función predictora del grupo, con nombre: predictor_v1_Grupo_X(), donde X es el número de su grupo

Información sobre las variables incluidas en el Dataset:

Feature	Descripción
canceled_job	Variable objetivo. Indica si la reserva fue cancelada o no llegó a ejecutarse.
hours_before_execution	Horas entre la creación de la reserva y el inicio programado de ejecución.
execution_year	Año del inicio programado de ejecución.
execution_month	Mes del inicio programado de ejecución.
execution_week	Semana del año del inicio programado de ejecución.
execution_day	Día del mes del inicio programado de ejecución.
peak_gpu_hours	Horas GPU reservadas en períodos de alta demanda.
offpeak_gpu_hours	Horas GPU reservadas en períodos de baja demanda.
num_primary_processes	Número de procesos principales asociados al workload.
num_auxiliary_processes	Número de procesos auxiliares asociados al workload.
num_background_processes	Número de procesos secundarios o de mantenimiento asociados al workload.
service_tier	Nivel de servicio contratado para la reserva.
client_region	País o región de procedencia del cliente.
client_type	Tipo de cliente que realiza la solicitud.
request_channel	Canal utilizado para realizar la reserva.
returning_client	Indica si el cliente ya había utilizado previamente la plataforma.
previous_job_cancellations	Cantidad de reservas canceladas previamente por el cliente.

successful_previous_jobs	Cantidad de reservas completadas exitosamente por el cliente.
reserved_gpu_type	Tipo de GPU solicitado originalmente.
assigned_gpu_type	Tipo de GPU finalmente asignado para la ejecución.
resource_modifications	Número de modificaciones realizadas sobre la reserva original.
reservation_commitment	Tipo de compromiso contractual asociado a la reserva.
orchestration_agent_id	Identificador del agente de orquestación asociado a la reserva.
organization_id	Identificador de la organización asociada a la reserva.
queue_wait_time	Tiempo de espera en cola antes de la asignación definitiva de recursos.
workload_profile	Perfil general o tipo de workload solicitado.
estimated_cost_per_hour	Costo estimado en USD por hora para el conjunto de recursos de la reserva.
reserved_storage_units	Cantidad de unidades de almacenamiento reservadas.
special_resource_requests	Número de solicitudes especiales asociadas a la reserva.